

Assignment No -3

Name : Atharv Gade

Roll No: COB413

Title: Implement Min ,Max ,Sum And Average Operation using Parellel Reduction.

Source Code with Output :

```
import random import
concurrent.futures import time

# Function for Parallel Reduction (find Min, Max, Sum, Average)
def parallel_reduce(arr, operation): chunk_size = len(arr) // 4 if
len(arr) >= 4 else 1

    def reduce_chunk(start, end):
        chunk = arr[start:end]    if
        operation == 'min':        return
        min(chunk)    elif operation ==
        'max':        return max(chunk)
        elif operation == 'sum':    return
        sum(chunk)    elif operation ==
        'average':    return sum(chunk) /
        len(chunk)

    else:        raise ValueError("Invalid operation. Choose 'min', 'max', 'sum', or
        'average'.")

    with concurrent.futures.ThreadPoolExecutor() as executor:
        futures = []    for i in range(0, len(arr), chunk_size):
        end = min(i + chunk_size, len(arr))
        futures.append(executor.submit(reduce_chunk, i, end))

    results = [future.result() for future in concurrent.futures.as_completed(futures)]
```

```

        if operation == 'min':      return
min(results)    elif operation == 'max':
return max(results)    elif operation == 'sum':
return sum(results)    elif operation ==
'average':      total_sum =
parallel_reduce(arr, 'sum')      return
total_sum / len(arr)

# Function to measure execution time def
measure_time(func, arr, operation):
start_time = time.time()    result = func(arr,
operation)    end_time = time.time()    return
result, end_time
- start_time

# Main function def main():    try:      n = int(input("Enter
the number of elements in the list: "))      if n <= 0:
print("Number of elements must be greater than 0.")
return    except ValueError:      print("Invalid input.
Please enter a valid integer.")
return

arr = [random.randint(1, 100) for _ in range(n)]    print(f"\nGenerated list
of {n} elements (showing first 10): {arr[:10]} ...\n")

# Perform all four operations    for operation
in ['min', 'max', 'sum', 'average']:      print(f"Performing
{operation.capitalize()}
operation using Parallel Reduction...")      result,
time_taken = measure_time(parallel_reduce, arr,
operation)      print(f"{operation.capitalize()} result:
{result}")      print(f"Execution Time:

```

```
{time_taken:.6f} seconds\n")
```

```
# Run the main function if __name__
```

```
== "__main__":
```

```
main()
```

Output :

```
Enter the number of elements in the list: 10
```

```
Generated list of 10 elements (showing first 10): [47, 66, 15, 63, 98, 9, 91, 78, 45, 43] ...
```

```
Performing Min operation using Parallel Reduction...
```

```
Min result: 9
```

```
Execution Time: 0.002025 seconds
```

```
Performing Max operation using Parallel Reduction...
```

```
Max result: 98
```

```
Execution Time: 0.001047 seconds
```

```
Performing Sum operation using Parallel Reduction...
```

```
Sum result: 555
```

```
Execution Time: 0.000758 seconds
```

```
Performing Average operation using Parallel Reduction...
```

```
Average result: 55.5
```

```
Execution Time: 0.001673 seconds
```